RAG Pipeline Architecture

Warren Buffett Intelligence Hub — Chatbot (Part 1)

Table of Contents

1. Overview
2. System Entry Point — The ask() Function
3. Singleton Model Loading and Caching
4. Query Classification — Stock vs. Regular
5. Follow-up Query Enrichment
6. Hybrid Retrieval — FAISS Semantic Search
7. Hybrid Retrieval — BM25 Keyword Search
8. Reciprocal Rank Fusion
9. Cross-Encoder Reranking
10. Context Construction
11. Prompt Engineering
12. Generation Backend — Groq Llama 3
13. Generation Backend — FLAN-T5 (Local)
14. Response Formatting and Source Citations
15. Conversation Memory
16. API Layer — FastAPI
17. End-to-End Flow Summary

---

1. Overview

The RAG (Retrieval-Augmented Generation) pipeline is the online query-answering engine of the Warren Buffett Intelligence Hub. It is responsible for taking a user's natural language question, finding the most relevant passages from the pre-built knowledge base, and generating a grounded, coherent answer using a large language model.

Unlike a pure language model that answers from parametric memory (training data), RAG grounds every answer in retrieved passages from the curated corpus of Berkshire Hathaway shareholder letters and ~6,000 hand-curated Buffett Q&A pairs. This architecture has several important advantages:

- **Factual grounding:** Answers cite actual documents, reducing hallucination.
- **Domain specificity:** The retrieval step filters the model's attention to Buffett-relevant knowledge, even if the underlying LLM was trained on much broader data.
- **Transparency:** Every answer is accompanied by source citations that allow the user to trace which documents informed the response.
- **Updatability:** The knowledge base can be updated by re-running `ingest.py` without retraining any model.

The RAG pipeline is implemented in `rag_chain.py` at the project root, with retrieval logic in `retriever.py` and the API layer in `server.py`.

---

2. System Entry Point — The ask() Function

The entire RAG pipeline is encapsulated in a single public function:

```
def ask(question, history=None):
```

Parameters:

- `question` (str): The user's natural language question.
- `history` (list, optional): A list of prior conversation turns, each a dict with `"role"` (`"user"` or `"assistant"`) and `"content"` keys. Used to support multi-turn conversation.

Return value:

The function returns a dictionary with the following keys:

```
{
    "answer":     str,          # The generated answer text
    "sources":    list[dict],   # Retrieved documents with source labels and page
numbers
    "stock_data": dict          # Only present if the question was about a stock
ticker
}
```

This single-function interface means the FastAPI server, the legacy Streamlit UI, and any test scripts all interact with the RAG pipeline through one consistent entry point. The internal complexity — retrieval, reranking, prompt building, generation — is completely encapsulated.

---

3. Singleton Model Loading and Caching

Loading large ML models is expensive. `rag_chain.py` uses module-level singletons to ensure models are loaded exactly once per server process:

```
_model = None
_tokenizer = None
_groq_client = None
```

The `get_llm()` function is called at the start of every `ask()` invocation. On the first call, it inspects the environment for a `GROQ_API_KEY`:

```
def get_llm():
    global _model, _tokenizer, _groq_client
    if _model is None and _groq_client is None:
```

```
        groq_key = os.environ.get("GROQ_API_KEY")
        if groq_key:
            from groq import Groq
            _groq_client = Groq(api_key=groq_key)
        else:
            _tokenizer = AutoTokenizer.from_pretrained(LOCAL_MODEL)
            _model = AutoModelForSeq2SeqLM.from_pretrained(LOCAL_MODEL)
```

- If `GROQ_API_KEY` is present in the environment (loaded from `.env` via `python-dotenv`), a `Groq` client is instantiated. Subsequent calls return this cached client without re-initializing.
- If no API key is present, the FLAN-T5 model and tokenizer are loaded from HuggingFace into memory. At ~900MB, this takes 30–60 seconds on first load but is cached in `_model` and `_tokenizer` for all subsequent requests.

Similarly, `retriever.py` uses module-level singletons for the FAISS vectorstore, BM25 index, and cross-encoder reranker:

```
_vectorstore = None
_bm25_index = None
_bm25_docs = None
_reranker = None
```

These are loaded lazily on first access and cached for the lifetime of the server process. Combined, this means the server pays a one-time startup cost on the first request but responds quickly to all subsequent queries.

---

4. Query Classification — Stock vs. Regular

Before retrieval begins, `ask()` determines whether the user's question is about a specific stock:

```
if is_stock_query(question):
    ticker = extract_ticker(question)
    if ticker:
        stock_data = fetch_stock_data(ticker)
        if stock_data:
            stock_summary = format_stock_summary(stock_data)
```

`is_stock_query()` and `extract_ticker()` are defined in `stock_analysis.py`. They use a combination of:

- **Keyword matching:** Checks for terms like `"stock"`, `"ticker"`, `"shares"`, `"invest in"`, `"buy"`, `"P/E"`, `"ROE"`, etc.
- **$TICKER pattern detection:** Recognizes `$AAPL`, `$MSFT`-style references.
- **Company name alias lookup:** Maps 50+ company names (e.g., `"Apple"`, `"Microsoft"`, `"Coca-Cola"`) to their ticker symbols.
- **Uppercase word heuristics:** Short uppercase tokens like `AAPL`, `TSLA`, `BRK.B` are tested against a known ticker list.

If the query is classified as a stock query and a ticker is successfully extracted, `fetch_stock_data()` is called to retrieve live financial data from Yahoo Finance via the `yfinance` library. This data includes price, P/E ratio, return on equity, profit margins, debt-to-equity ratio, 52-week range, and 1-year/5-year return figures.

Why Classification Matters

A stock query requires a fundamentally different pipeline branch:

Aspect	Regular Query	Stock Query
Retrieval search string	User's question	Fixed investment criteria query
Prompt template	General Buffett Q&A prompt	Stock analysis prompt with live data
Data sources in response	Retrieved documents only	Live stock data + retrieved documents

For stock queries, the retrieval search string is overridden:

```
if stock_data:
    search_query = "investment criteria value investing margin of safety return on equity"
```

This fixed query is used instead of the user's raw question to retrieve Buffett's general investment principles — the reasoning being that for stock analysis, the relevant context is always about valuation, moats, and returns, regardless of which specific company is being asked about.

---

5. Follow-up Query Enrichment

For multi-turn conversations, short follow-up questions can be ambiguous in isolation. A question like `"Why?"` or `"Tell me more"` is too sparse to retrieve meaningful documents on its own.

The pipeline detects short follow-up questions and enriches the search query with context from the most recent assistant response:

```
if history:
    last_assistant = [m for m in history if m["role"] == "assistant"]
    if last_assistant and len(question.split()) < 8:
        search_query = f"{last_assistant[-1]['content'][:200]} {question}"
```

If the question has fewer than 8 words and there is prior assistant output in the conversation history, the last 200 characters of the most recent assistant message are prepended to the search query. This enriched query gives the FAISS and BM25 retrieval steps enough context to surface documents relevant to the ongoing conversation topic rather than treating the follow-up as a standalone query.

---

6. Hybrid Retrieval — FAISS Semantic Search

The first retrieval component is a dense vector similarity search over the FAISS index built during ingestion.

```
faiss_results = vectorstore.similarity_search(query, k=k)
```

The user's query (or enriched search query) is embedded using the same `paraphrase-MiniLM-L6-v2` model that was used during ingestion, producing a 384-dimensional query vector. FAISS then performs an exact nearest-neighbor search over all document vectors in the index using L2 distance (equivalent to cosine similarity due to vector normalization).

FAISS returns the top-`k` most semantically similar documents. Because the embedding model was trained for paraphrase detection, it can match a user's question to a document even when the phrasing is very different — for example, matching `"How does Buffett think about competitive advantage?"` to a document about `"economic moat"` because these phrases are semantically close in the embedding space.

Strengths of FAISS retrieval:

- Captures semantic and paraphrase similarity
- Works well for conceptual questions
- Handles synonyms and restatements naturally

Weaknesses of FAISS retrieval:

- Can miss documents that contain exact keywords but are semantically distant
- Embedding quality degrades on highly specific proper nouns or numerical facts

This is why FAISS is combined with BM25 rather than used alone.

---

7. Hybrid Retrieval — BM25 Keyword Search

The second retrieval component is BM25, a classical term-frequency-based keyword search algorithm.

```
query_tokens = tokenize(query)
bm25_scores = bm25.get_scores(query_tokens)
bm25_top_indices = bm25_scores.argsort()[-k:][::-1]
bm25_results = [bm25_docs[i] for i in bm25_top_indices if bm25_scores[i] > 0]
```

The query is tokenized using the same whitespace/lowercase tokenizer used during index construction. BM25 then scores every document in the corpus based on how many query tokens appear in the document, weighted by their inverse document frequency (rarer terms get higher weight) and normalized for document length.

Only documents with a BM25 score greater than zero are returned — this filters out documents that contain none of the query terms, which would otherwise be ranked arbitrarily.

Strengths of BM25 retrieval:

- Precise keyword matching — finds documents containing exact terms

- Effective for queries with specific names (companies, people, places), years, or technical terms
- Computationally inexpensive compared to dense search

Weaknesses of BM25 retrieval:

- Cannot handle paraphrase or synonymy
- A document about `"economic moat"` will not be found by a query about `"competitive advantage"`
- Sensitive to query phrasing

The complementary nature of FAISS (semantic) and BM25 (lexical) is what makes hybrid search significantly more robust than either method alone.

---

8. Reciprocal Rank Fusion

After both FAISS and BM25 return their top-`k` results independently, the two ranked lists are merged using Reciprocal Rank Fusion (RRF):

```
rrf_constant = 60  # standard RRF constant

for rank, doc in enumerate(faiss_results):
    rrf_scores[doc_id] = rrf_scores.get(doc_id, 0) + 1.0 / (rrf_constant + rank +
1)

for rank, doc in enumerate(bm25_results):
    rrf_scores[doc_id] = rrf_scores.get(doc_id, 0) + 1.0 / (rrf_constant + rank +
1)
```

How RRF Works

Each document receives an RRF score contribution from each ranked list it appears in:

```
RRF_score(doc, list) = 1 / (k + rank)
```

where `k = 60` is a constant that dampens the influence of top-ranked documents, and `rank` is the document's 1-indexed position in that list.

A document's total RRF score is the sum of its contributions from all lists. Documents that rank highly in **both** FAISS and BM25 receive a high combined score. Documents that rank highly in only one list receive a moderate score. Documents not appearing in either list receive zero.

Why RRF over Score Normalization

A naive approach to merging ranked lists would be to normalize and sum the raw similarity scores from FAISS and BM25. However, this is problematic because the score distributions of the two methods are incompatible — FAISS cosine similarities range from 0 to 1, while BM25 scores are unbounded and depend on corpus

statistics. RRF sidesteps this problem entirely by operating only on rank positions, making it robust to differences in score scale.

The `rrf_constant = 60` is the standard value from the original RRF paper (Cormack et al., 2009). It controls how much weight is given to rank position versus mere presence in the list — higher values produce a more democratic fusion.

After RRF scoring, documents are sorted by their combined RRF score and the top-`k_retrieve = 10` candidates are passed to the reranker.

---

9. Cross-Encoder Reranking

The final retrieval step is reranking the 10 RRF-merged candidates using a cross-encoder model:

```
RERANKER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"

pairs = [(query, doc.page_content) for doc in docs]
scores = reranker.predict(pairs)
scored_docs = sorted(zip(scores, docs), key=lambda x: x[0], reverse=True)
return [doc for _, doc in scored_docs[:top_n]]
```

Bi-Encoder vs. Cross-Encoder

The FAISS retrieval uses a **bi-encoder** architecture — the query and each document are embedded independently and similarity is measured by vector distance. This is fast (the document vectors are pre-computed at index time) but less accurate, because the query and document representations are computed in isolation without attending to each other.

A **cross-encoder** processes the query and document *together* as a single input sequence:

```
[CLS] query [SEP] document [SEP]
```

Both the query and the document attend to each other through all transformer layers. This joint processing produces a much more accurate relevance score, because the model can reason about how specific parts of the document answer specific parts of the query.

The cross-encoder used here is `cross-encoder/ms-marco-MiniLM-L-6-v2`, trained on the MS MARCO passage ranking dataset — a large-scale information retrieval benchmark derived from real Bing search queries and human-judged relevant passages.

Why Not Use the Cross-Encoder for All Documents

Cross-encoder inference is significantly slower than FAISS because it cannot pre-compute document embeddings — every query requires a fresh forward pass for every (query, document) pair. Running the cross-encoder over all 10,000+ documents for every query would be prohibitively slow.

The two-stage retrieval architecture solves this: FAISS and BM25 cheaply narrow the candidate set to 10 documents, and then the cross-encoder applies its more accurate (but slower) scoring only to those 10 candidates. This is the standard industry approach to production-scale retrieval systems.

After reranking, the top `k_final = 5` documents (as configured by `TOP_K` in `config.py`) are returned as the final retrieval result.

---

10. Context Construction

The 5 reranked documents are assembled into a single context string for the prompt:

```
def build_context(docs):
    context_parts = []
    for i, doc in enumerate(docs, start=1):
        meta = doc.metadata
        source = meta.get("source", "unknown")
        page = meta.get("page")
        header = f"[Context {i}] Source: {source}"
        if page is not None:
            header += f" (page {page + 1})"
        context_parts.append(f"{header}\n{doc.page_content}")
    return "\n\n".join(context_parts)
```

Each document is prefixed with a header that identifies its source and page number (for PDF-derived documents). The five blocks are separated by double newlines. This structured formatting helps the LLM distinguish between different retrieved passages and understand their provenance.

The source labels take the form:

- `shareholder_letter (page 42)` — for PDF-derived chunks
- `qa_risk_management` — for Q&A pairs from the Risk Management CSV
- `qa_strategy_and_moats` — for Q&A pairs from the Strategy CSV
- `yahoo_finance` — for live stock data (appended to context in stock queries)

For the local FLAN-T5 model, only the top 3 documents (not all 5) are used to fit within the 512-token input limit:

```
short_context = build_context(docs[:3])
```

---

11. Prompt Engineering

The system uses four distinct prompt templates, each tailored to a specific scenario.

11.1 Groq — Regular Question

```
def build_groq_prompt(question, context, chat_history=""):
    return f"""You are a Warren Buffett trader/investor chatbot built from team-prepared study material.
Answer the user's question using ONLY the retrieved context below.
{history_block}
Rules:
- Do not mention "Context 1", "Context 2", or similar references.
- Do not say "according to the context".
- Give a direct, natural answer.
- Prefer 2-4 sentences.
- Be specific when the dataset supports specifics.
- If this is a follow-up question, use the conversation history to understand what the user is referring to.
- If the question asks how Buffett adapted or evolved, prioritize concrete historical changes in his strategy.
- Avoid vague summaries like "he learned from experience" unless no more specific answer is available.
- If the answer is not clearly supported by the retrieved material, say:
  "I don't have enough grounded information in the dataset to answer that confidently."

Retrieved Context:
{context}

User Question:
{question}"""
```

Key design decisions in this prompt:

- **Instruction to use ONLY retrieved context:** This is the core RAG constraint. It prevents the model from hallucinating answers from its parametric training data and forces grounding in the curated corpus.
- **Explicit citation style prohibition:** Telling the model not to say `"Context 1"` or `"according to the context"` produces more natural, conversational responses — the citations appear in the UI through the `sources` list in the response, not in the answer text.
- **Calibrated answer length:** `"Prefer 2-4 sentences"` targets a response length appropriate for a chat UI — detailed enough to be informative, brief enough to be readable.
- **Graceful fallback:** Instructing the model to explicitly admit uncertainty is critical in a domain-specific application. Without this instruction, models tend to produce confident-sounding but unfounded answers when the context doesn't support a clear response.
- **Evolution-specific instruction:** The explicit rule about Buffett's strategic evolution reflects a known failure mode observed during development — without it, the model tends to give vague `"he learned and grew"` answers instead of citing concrete historical shifts (e.g., the documented shift from Graham-style net-net investing to buying high-quality businesses at fair prices).

11.2 Groq — Stock Analysis

For stock queries, a separate prompt template incorporates live financial data alongside the retrieved Buffett principles:

```
def build_stock_groq_prompt(question, context, stock_summary):
    return f"""You are a Warren Buffett-style investment analyst.
...
Stock Data & Backtest Results:
{stock_summary}

Buffett's Investment Principles (from dataset):
{context}

User Question:
{question}"""
```

This prompt instructs the model to:

- Reference actual numerical metrics (P/E, ROE, margins) from the stock data
- Mention how backtesting strategies performed on the stock
- Apply Buffett's known criteria: margin of safety, economic moat, ROE, low debt, consistent earnings
- Give a clear `"would Buffett be interested?"` assessment
- Acknowledge the limitations of backtesting (past performance, no future prediction)

The two-source structure (live data + retrieved principles) is made explicit in the prompt so the model understands it has both quantitative evidence and qualitative Buffett philosophy to work with.

11.3 Local FLAN-T5 — Regular Question

```
def build_local_prompt(question, context):
    return (
        "Use the following context to give a detailed answer to the question. "
        "If the answer is not in the context, say so.\n\n"
        f"Context:\n{context}\n\n"
        f"Question: {question}\n\n"
        "Detailed answer:"
    )
```

This is a simpler, shorter prompt appropriate for FLAN-T5's 512-token input limit. The `"Detailed answer:"` suffix is a standard instruction-following prompt pattern for FLAN-T5, which was trained on instruction-formatted datasets.

11.4 Local FLAN-T5 — Stock Analysis

A similarly compact prompt for stock analysis under the FLAN-T5 token constraint:

```
def build_stock_local_prompt(question, context, stock_summary):
    return (
        "Analyze this stock using Warren Buffett's investment principles.\n\n"
        f"Stock Data:\n{stock_summary}\n\n"
        f"Buffett's Principles:\n{context}\n\n"
```

```
        f"Question: {question}\n\n"
        "Analysis:"
    )
```

---

12. Generation Backend — Groq Llama 3

When a `GROQ_API_KEY` is configured, the generation step calls Groq's cloud inference API:

```
response = groq_client.chat.completions.create(
    model=GROQ_MODEL,   # "llama-3.3-70b-versatile"
    messages=[{"role": "user", "content": prompt}],
)
answer = response.choices[0].message.content.strip()
```

Model: `llama-3.3-70b-versatile` (Meta's Llama 3.3, 70 billion parameter variant)

Groq provides hardware-accelerated inference on their Language Processing Units (LPUs), delivering very low latency on large models. The 70B parameter model produces significantly higher quality answers than the local FLAN-T5-base model — it handles nuanced reasoning, multi-sentence synthesis, and instruction-following much more reliably.

Key characteristics of Llama 3.3 70B on Groq:

Property	Value
Parameters	70 billion
Architecture	Transformer decoder (Llama 3 family)
Context window	128,000 tokens
Inference latency	~1–2 seconds via Groq LPU
Input format	OpenAI-compatible chat completions API

Because the context window is 128K tokens, there is no practical input length constraint for this generation path — the full 5-document context is always used, and conversation history is always included.

The `"versatile"` model variant is used rather than a `"preview"` or `"instruct-only"` variant because it performs well on both instruction-following tasks (the structured prompt rules) and open-ended generation tasks (synthesizing a natural-sounding answer from multiple retrieved passages).

---

13. Generation Backend — FLAN-T5 (Local)

When no `GROQ_API_KEY` is available, the pipeline falls back to a locally-running FLAN-T5-base model:

```
inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512)
outputs = model.generate(**inputs, max_new_tokens=256)
answer = tokenizer.decode(outputs[0], skip_special_tokens=True).strip()
```

Model: `google/flan-t5-base`

FLAN-T5 (Fine-tuned Language Net - Text-To-Text Transfer Transformer) is a sequence-to-sequence transformer developed by Google. It was fine-tuned on over 1,800 NLP tasks expressed as natural language instructions, making it capable of following prompts in a structured way despite its relatively small size.

Key characteristics:

Property	Value
Parameters	250 million
Architecture	Encoder-decoder T5
Max input tokens	512
Max output tokens	256 (configured)
Model size on disk	~900MB
Inference hardware	CPU (no GPU required)

Critical limitation — 512-token input:

FLAN-T5-base has a hard 512-token maximum input length. For context, 512 tokens is approximately 380 words, which is just enough for a short prompt instruction, a 3-document context, and the question. The tokenizer's `truncation=True` parameter silently clips any input that exceeds this limit — if the context is too long, it will be truncated and the model will answer from incomplete information.

This is why the FLAN-T5 code path uses only the top 3 retrieved documents (rather than 5) and a simpler, more compact prompt template.

Why include a local fallback at all?

The local FLAN-T5 path allows the application to function without any API keys or internet access. For development and demonstration purposes where the full Groq integration is not available (e.g., no API key, rate limits, or offline environments), FLAN-T5 provides a degraded-but-functional answer generation capability.

In production or demonstration settings, Groq with Llama 3 is always preferred.

---

14. Response Formatting and Source Citations

After generation, `ask()` constructs the sources list for the response:

```
sources = []
if stock_data:
    sources.append({
        "content": stock_summary,
        "source": "yahoo_finance",
        "page": None,
    })
for doc in docs:
    sources.append({
        "content": doc.page_content,
        "source": doc.metadata.get("source", "unknown"),
        "page": doc.metadata.get("page"),
    })
```

Each source entry contains:

- `content`: The raw text of the retrieved passage (displayed in the "Sources" panel of the chat UI)
- `source`: The source label (`"shareholder_letter"`, `"qa_risk_management"`, `"yahoo_finance"`, etc.)
- `page`: The page number within the PDF, if applicable (null for CSV-derived documents)

In the React frontend's chat UI, these sources are rendered as expandable citation cards below each assistant message, enabling users to read the exact passages that informed the answer and verify that the response is grounded in the retrieved material.

The final return value assembles everything:

```
result = {"answer": answer, "sources": sources}
if stock_data:
    result["stock_data"] = stock_data
return result
```

---

15. Conversation Memory

Multi-turn conversation is supported through the `history` parameter and the `format_chat_history()` function:

```
def format_chat_history(history):
    if not history:
        return ""
    lines = []
    for msg in history[-4:]:  # last 4 messages (2 turns)
        role = "User" if msg["role"] == "user" else "Assistant"
        lines.append(f"{role}: {msg['content']}")
    return "\n".join(lines)
```

Only the **last 4 messages** (2 complete user/assistant turns) are included in the prompt. This is a deliberate design decision:

- **Token budget:** Including unlimited history would eventually exceed FLAN-T5's 512-token limit and make Groq prompts unnecessarily long.
- **Recency bias:** In practice, the most recent 2 turns contain the context needed to resolve follow-up questions. Older turns are rarely needed.
- **Performance:** Keeping history short means the retrieval enrichment heuristic (detecting short follow-up questions) remains reliable — older, unrelated conversation turns are not mixed into the search query.

The formatted history is included in the Groq prompt under a `"Previous conversation:"` block, placed between the system instructions and the retrieved context. This placement gives the model access to conversation context before it reads the documents, which helps it interpret ambiguous follow-up questions correctly.

Note: Conversation memory is stateless on the server side. The full history is passed by the client (the React frontend maintains the conversation state in React `useState`) with every request. The server does not store any session data.

---

16. API Layer — FastAPI

The RAG pipeline is exposed to the React frontend through a FastAPI server defined in `server.py`. The server runs at `http://localhost:8000` and CORS is configured to allow requests only from the React dev server at `http://localhost:5173`.

16.1 Chat Endpoint

```
@app.post("/api/chat")
def chat(req: QuestionRequest):
    from rag_chain import ask
    return ask(req.question)
```

The `QuestionRequest` Pydantic model:

```
class QuestionRequest(BaseModel):
    question: str
    history: list[ChatMessage] = []
```

The `ask()` function is imported inside the endpoint function rather than at module level. This deferred import means the heavy model loading (FLAN-T5 or Groq client initialization) does not happen at server startup — it happens on the first request. This keeps server startup fast.

16.2 Buffett Take Endpoints

Two additional endpoints invoke the RAG pipeline for non-chat use cases:

`POST /api/stock/buffett-take` — Used by the Stock Trading tab's "Get Warren Buffett's Take" button. Receives a stock symbol and lists of passing/failing Buffett criteria, constructs a structured question, and calls `ask()`:

```
question = (
    f"Using Warren Buffett's principles, evaluate {req.symbol}. "
    f"Criteria passed: {passing_str}. "
    f"Criteria failed: {failing_str}. "
    f"Based on these fundamentals, provide a Buffett-style assessment."
)
result = ask(question)
return {"answer": result["answer"]}
```

`POST /api/backtest/buffett-take` — Used by the Backtesting tab after a strategy is run. Receives backtest performance metrics and asks the RAG system what Buffett would think of the trading approach:

```
question = (
    f"A {req.strategyName} strategy on {req.symbol} produced: "
    f"total return {req.totalReturn * 100:.1f}%, Sharpe ratio
{req.sharpeRatio:.2f}, "
    f"max drawdown {req.maxDrawdown * 100:.1f}%, win rate {req.winRate *
100:.1f}%, "
    f"{req.numTrades} trades. "
    f"From Warren Buffett's perspective, what does he think of this trading
approach and results?"
)
```

Both endpoints share the same underlying RAG infrastructure — the same `ask()` function, the same retrieval pipeline, the same generation backend.

16.3 Health Check

```
@app.get("/api/health")
def health():
    return {"status": "ok"}
```

A lightweight liveness endpoint that the React frontend can poll to determine whether the backend is running.

---

17. End-to-End Flow Summary

```
User types a question in the React chat UI
    |
    ▼
POST /api/chat  {question, history}
```

```
    │
    ▼
ask(question, history)
    │
    ├── get_llm()  [load Groq client or FLAN-T5, cached after first call]
    │
    ├── is_stock_query(question)?
    │     ├── YES → extract_ticker() → fetch_stock_data() → format_stock_summary()
    │     │         search_query = "investment criteria value investing ..."
    │     └── NO  → search_query = question
    │                 (enriched with last assistant message if short follow-up)
    │
    ├── retrieve(search_query, k_retrieve=10, k_final=5)
    │     ├── FAISS semantic search   → top 10 by cosine similarity
    │     ├── BM25 keyword search     → top 10 by TF-IDF score
    │     ├── Reciprocal Rank Fusion  → merged & re-ranked list of ≤20 candidates
    │     └── Cross-encoder rerank   → top 5 by joint query-document scoring
    │
    ├── build_context(docs)  → structured context string with source headers
    │
    ├── format_chat_history(history[-4:])  → last 2 turns for conversation memory
    │
    ├── Build prompt
    │     ├── Groq + stock  → build_stock_groq_prompt(question, context,
stock_summary)
    │     ├── Groq + chat   → build_groq_prompt(question, context, chat_history)
    │     ├── Local + stock → build_stock_local_prompt(question, context[:3],
stock_summary)
    │     └── Local + chat  → build_local_prompt(question, context[:3])
    │
    ├── Generate answer
    │     ├── Groq path  → groq_client.chat.completions.create(model="llama-3.3-
70b-versatile")
    │     └── Local path → tokenizer + model.generate(max_new_tokens=256,
truncation to 512)
    │
    └── Return {answer, sources, stock_data?}
            │
            ▼
    React frontend renders:
        - Answer text in chat bubble
        - Expandable source citation cards
        - Stock data panel (if stock query)
```

Document prepared for CSYE7380 Final Project presentation — Warren Buffett Intelligence Hub (Part 1: RAG Chatbot)